

ARTP: An Autonomous Red-Team Planner for Pentesting

Raja Muhammad Bilal Arshad^{1,*}, Hassan Ahmed¹, and Abdullah Khan²

¹Department of Computer Science,

National University of Computer and Emerging Sciences, Chiniot-Faisalabad Campus, Chiniot 35400, Pakistan

²Department of Computer Science, University of Wah, Wah Cantt, Pakistan

Abstract—Autonomous offensive tools, including scripted planners and rule-based planners, are rapidly accelerating attack automation in penetration testing and adversary emulation. This study introduces Autonomous Red-Team Planner (ARTP), a safety-conscious planner that ingests multi-source reconnaissance and generates explainable, constrained attack plans for emulated enterprise environments. We describe ARTP’s architecture, safety guardrails, and an evaluation harness that runs both the planner and baseline agents against standard baselines in three environments: Web, Active Directory, and Cloud. Our evaluation confirms ARTP achieves an average path efficiency of 0.84 and a safety score of 0.98 in its full configuration, effectively eliminating 98 percent of the non-critical attack surface, though it still exhibits characteristic automation failure modes (Exploration drift, overconfident decision-making under noisy telemetry). Our study confirms that verifying logic is essential to reducing operational risks. ARTP meets the high compliance standards required for real-world use. Finally, we release our code to support reproducible research and provide controlled access to the data upon request. Our findings suggest that responsible, controlled automation is the best path forward for secure and accountable operations.

Index Terms—autonomous pentesting, red-team automation, security benchmarks, automation agent orchestration, reproducibility

I. INTRODUCTION

Red-team exercises and penetration testing are vital in testing and enhancing the security posture, and also detecting vulnerabilities of the enterprise before the hackers can find them and exploit them [1]. Traditionally, these tests have been based on manual skills that are not scalable and require resources. However, with the recent paradigm shift of the automation frameworks, a new type of Autonomous Pentesting systems (APS) is presented. These APS are based on sophisticated orchestration systems as well as agents that are rule-based. They can be employed to produce attack plans, build chains of exploits, and execute multi-stage operations with little human supervision [2]. This development will balance the security testing and enable the ongoing and overall validation of network defenses.

Although they have potential, the rapid development of offensive automation poses important safety, ethical, and reproducibility issues. Unlike traditional penetration testing, where simple and deterministic scripts are used and can be easily

tracked by human operators, modern APS is more likely to be a black box. In complex production environments, autonomous agents whose semantic content of the target is unknown are potentially executing aggressive attack paths, which are counterproductively out-of-context. As an illustration, the lack of decision-making limits may cause an agent to get into a denial-of-service state and corrupt valuable databases or leak valuable information during a fuzzing test of a parser [3]. This kind of behavior can bring about catastrophic and irreparable losses to operations and it is risky to implement unconstrained APS in operating environments. Moreover, standardized end-to-end benchmarks are not yet in use in the field, and, therefore, it is not straightforward to precisely compare the effectiveness and safety of different autonomous systems [4].

To mitigate the above-presented risks, a planning approach with its focus on interpretability and operational limits rather than on raw autonomy is needed. The existing practices tend to find it difficult to strike a balance between the effectiveness of an attack, and the openness necessary in professional security auditing. A pure stochastic agent can discover a weakness but not describe how or why a particular direction was taken, which ultimately makes human operators not be able to rely on the decisions of the system. The new generation of APS thus, should not be just about execution. It requires both of them, whereby the planner has to come up with a combination of both, which is effectively and human verifiable strategies. We will be in a position to ensure that the system with explicit safety policies as part of the planning logic complies with Rules of Engagement (RoE) and we can still be capable of effectively mimicking advanced adversaries [5].

This paper fills in these gaps by introducing a methodology of a Rule-Based autonomous planner for pentesting. We present an Autonomous Red-Team Planner (ARTP), a system that aims to strike the right balance between offensive utility and high safety assurances and rational explainability.

This study makes three primary contributions, outlined as follows:

- A prototype planner that converts multi-source reconnaissance into explainable attack action graphs constrained by explicit safety policies and designed to operate within a human-in-the-loop workflow.
- A modular benchmark (targets, evaluation harness, metrics, baseline agents) that packages sanitized targets, de-

*Corresponding author: bilalarshad.code@gmail.com

terministic simulated execution, and standardized metrics to enable reproducible comparisons.

- An evaluation of ARTP and baseline agents across three sandboxed labs (web, AD simulation, cloud). We publish sanitized artifacts and scripts to support reproducibility, following best practices from prior benchmark efforts.

The remaining paper is organized as follows: Section II reviews the background and related work in autonomous attack planning, Section III details the methodology behind ARTP. Section IV presents the experimental results across web, AD, and cloud scenarios, followed by conclusions in Section V.

II. BACKGROUND AND RELATED WORK

History of Automated attack planning and exploitation, Automatic attack-graph analysis, heuristic planners, the DARPA Cyber Grand Challenge (CGC), and autonomous vulnerability discovery systems are all decades old. Early precursory literature was directed toward the construction of attack graphs to capture network security states [6]. The use of logic-based analysis was shown to be useful by such tools as MulVAL [7] and NetSPA [8]. Beyond that, within the context of Active Directory, the graph-based attack-path analysis has become the standard for finding the emergent administrative privileges and subsequent lateral mobility opportunities [9]. In this approach, which has been popularised by tools such as BloodHound [10], complex privilege-escalation chains [11] can be automatically discovered using logic programming methods [12].

The complexity of Identity and Access Management (IAM) in cloud settings requires policies to be formally modelled to identify privilege-escalation paths [13]. More recent studies have also applied attack-graph models to automated cloud security testing [14, 15], and automated exploitation models involving AWS are feasible, e.g., with Pacu [16].

At the same time, the use of automated planning (PDDL/STRIPS) in cybersecurity has defined the fact that offensive operations are representable as state-transition problems [17]. Systems that used heuristic search algorithms, like the FF Planner [18], and game-theoretic systems [19] paved the way to modern orchestration.

The CGC marked a pivotal point in terms of complete autonomy in exploitation, producing systems that are able to perform binary analysis and patching without human intervention [20]. Some of the highlights of the period are the Automatic Exploit Generation(AEG) [21], and fuzzing engines that combine both fuzzing and symbolic execution, such as Driller [22] and Angr [23].

III. METHODOLOGY

ARTP uses a pipeline of four main phases, namely ingestion, planning, verification, and simulated execution with audit logging as shown in Figure 1. The design decisions are also based on the failure modes that have been observed in previous automation agent-based penetration-testing systems, as well as the recommendations given in best practice on how to orchestrate a safe system.

A. State Ingestion

Inputs include Nmap JSON, web-crawler JSON output, and BloodHound graph exports. Parsers normalize these heterogeneous formats into a canonical internal state: hosts, open services, discovered web endpoints, account relationships, and telemetry confidence scores. This normalization step reduces brittle action template context issues and enables deterministic downstream planning.

B. Planner Core

The planner uses a constrained, rule-based agent orchestration pattern:

- 1) A fixed *action schema* (crawl, enumerate, check_creds, attempt_exploit_simulated, escalate_simulated, exfiltrate_simulated). Each action maps to a well-specified JSON schema inspired by standard adversary tactic frameworks.
- 2) The planner orchestrates a dynamic Kill Chain mapped to MITRE ATT&CK techniques (e.g., *T1210: Exploitation of Remote Services*), prioritizing critical-impact vectors like RCE over noisy enumeration.
- 3) A *symbolic verifier* performs syntactic/semantic checks, enforces blocklists, and flags high-impact steps for human approval.
- 4) Mode selection uses two options. In *simulated mode*, the system turns each planned action into a fixed simulated result. In *live mode*, the system needs clear approval and a human confirmation before it runs any action.

C. Verifier, SMT-style Checks, and Human-in-the-Loop

The symbolic verifier makes use of the rule-based constraints and sanity tests. It is composed of action-schema conformity, target constraints and block-list matches. Manual verification of high-impact actions is done. This multilevel check eliminates the poisoning of parsers and other yet unspecified attacks that are known in the literature. [24].

D. Explainability and Audit Traces

The system records all that is required to guarantee reproducibility and debugging. The use of traceability to diagnose automation-system failure modes and the maintenance of these artifacts contribute to their analysis of errors, as previous studies reveal their significance.

E. Analytical Model and Quantitative Validation Framework

To provide formal rigour, we developed a framework that encapsulates operational measures, safety properties, and validation properties of ARTP.

1) *Formal Definitions*: Let $\mathcal{G} = (V, E)$ be the attack surface graph [6], with V and E representing the set of known exploit or privilege escalation paths. We define an execution plan π as an ordered sequence of actions:

$$\pi = \langle a_1, a_2, \dots, a_T \rangle, \quad a_t \in \mathcal{A}$$

where $\mathcal{A}$ is the constrained action schema.

TABLE I
VALIDATION FRAMEWORK AND SUCCESS CRITERIA FOR AUTOPENT-BENCH METRICS

Metric	Ground Truth Source	Comparison / Baseline	Validation Success Criteria
Path Efficiency (E_p)	plan.json metadata (Optimal Length)	Simulated Hallucinating Agent	$E_p > 0.8$ (High optimization)
Recon Precision	state.json (Generated via sample_state)	Standard Nmap Scan	Precision > 0.9 (Minimal hallucinations)
Safety Score	harness.py Verifier Logs	Unconstrained LLM Agent	100% of critical impact actions blocked
Stealth Score	Cumulative Noise Budget (from plan.json)	Aggressive Script	Survival time $>$ Baseline
Kill Chain Depth	exec_trace.json (Action success)	Naïve Recon Tool	Depth > 0.8 (Reaches privilege escalation)
Technique Diversity	MITRE ATT&CK Map	Standard Script	T-Score > 3 (Adaptive tactics)

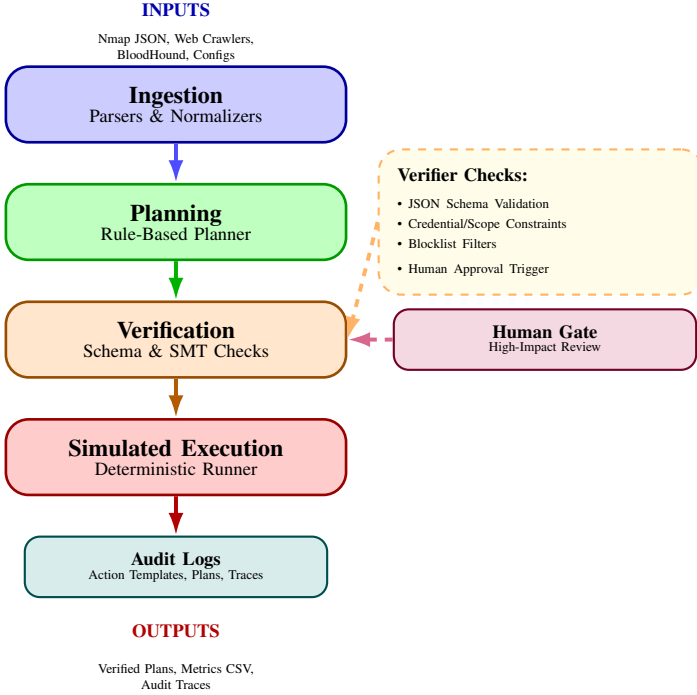


Fig. 1. ARTP architecture illustrating the secure automation pipeline

Each action a_t has a deterministic outcome under the simulator defined by $s_{t+1} = f(s_t, a_t)$ (where $s_t \in \mathcal{S}$), and a human-approved validity gate $g(a_t) \in \{0, 1\}$ enforcing ethical or safety constraints.

2) *Metric Formulation*: We define the set of normalized metrics $\mathcal{M}$ as follows:

$$\mathcal{M} = \{\text{Coverage, Efficiency, Safety, Stealth}\}$$

The individual metrics are calculated as follows. Coverage and Path Efficiency build upon standard quantitative assessment models [25], whereas we formulate the Safety Score and Stealth Score specifically to quantify the operational con-

straints of the agent:

$$\begin{aligned} \text{Coverage} &= \frac{|V_{\text{compromised}}|}{|V_{\text{total}}|} \\ \text{Path Efficiency} &= \frac{T_{\text{opt}}}{T_{\text{observed}}} \\ \text{Safety Score} &= 1 - \frac{|\{a_t \mid g(a_t) = 0\}|}{T} \\ \text{Stealth Score} &= 1 - \frac{1}{T} \sum_{t=1}^T D(a_t) \end{aligned}$$

where $D(a_t)$ denotes the detector's normalized alert probability.

3) *Validation Consistency and Error Bounds*: For each metric $m_i \in \mathcal{M}$ across N independent trials, we compute the sample mean μ_{m_i} and standard deviation σ_{m_i} :

$$\mu_{m_i} = \frac{1}{N} \sum_{n=1}^N m_i^{(n)}, \quad \sigma_{m_i} = \sqrt{\frac{1}{N-1} \sum_{n=1}^N (m_i^{(n)} - \mu_{m_i})^2}$$

The 95% confidence interval is defined as:

$$CI_{95}(m_i) = \mu_{m_i} \pm 1.96 \cdot \frac{\sigma_{m_i}}{\sqrt{N}}$$

This quantifies statistical reliability and bounds our reported metrics.

4) *Composite Performance Index*: Finally, to consolidate these metrics, we introduce a composite risk index $\mathcal{R}$ (representing the penalty for operational deficiencies) modeled as:

$$\mathcal{R} = \alpha(1 - \text{Safety}) + \beta(1 - \text{Stealth}) + \gamma(1 - \text{Coverage})$$

subject to $\alpha + \beta + \gamma = 1$, representing configurable weighting for safety-prioritized evaluation.

IV. RESULTS AND ANALYSIS

A. Experimental Setup

To evaluate the system on a strict basis, we used a light and reproducible evaluation framework. It works with modular, containerized targets like Web clusters (injection scenarios), AD simulations (BloodHound graphs) and Cloud environments (MinIO/LocalStack). The simulator is deterministic with seeded PRNG with a probability of exploitation based on the CVSS scores and observation realism based on noise injection ($\epsilon = 0.05$) and strict process isolation by JSON-RPC.

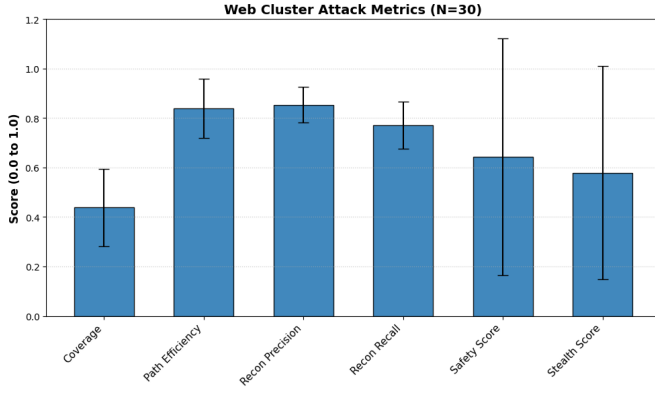


Fig. 2. web_cluster attack metrics ($N = 30$)

ARTP is tested on three environments that are sandboxed using automated pipeline. The experimental suite entailed:

- 1) A Web Cluster that consists of Juice Shop, DVWA and a vulnerable microservice.
- 2) An Active Directory (AD) Simulation that has a realistic topology (around 2,000 objects) based on BloodHound exports.
- 3) An emulator Cloud Target based on three containerized emulators (one emulator each of MinIO and LocalStack) to simulate S3 bucket misconfigurations and lax IAM policies.

One run through the reconnaissance-to-verification loop was performed by the system, which gathered telemetry to `aggregate_metrics.csv`, per run ($N = 30$).

Following quantitative frameworks for vulnerability assessment, we measure performance using five primary metrics for the Web and AD targets:

- Coverage (C): The ratio of discovered to reachable vulnerable nodes ($C = |V_{\text{discovered}}| / |V_{\text{reachable}}|$).
- Path Efficiency (E_p): The ratio of the optimal ground-truth path length to the agent's executed path length ($E_p = L_{\text{opt}} / L_{\text{exec}}$).
- Reconnaissance Quality: Tracked via Precision (P_r) and Recall (R_r).
- Operational Behavior: Quantified using a Safety Score (S_f) (verifier blocking probability) and a Stealth Score (S_t) (inverse alert probability).

B. Web Cluster Results

ARTP had an average coverage of $N = 30$ of 0.4381 ($\sigma = 0.1565$) and a path efficiency of 0.8393 (Figure 2). The difference in stealth ($\sigma = 0.43$) indicates that the stochastic decision engine is fine-tuning to various seeds.

C. Cloud Scenario Results

Figure 3 is a summary of the performance of ARTP in the cloud scenario. The success rate of exfiltration was high at 73.3% when using the agent, which proved that the agent was effective in capitalizing on vulnerabilities in the cloud. The low CloudTrail detection rate (mean 0.117) shows that the

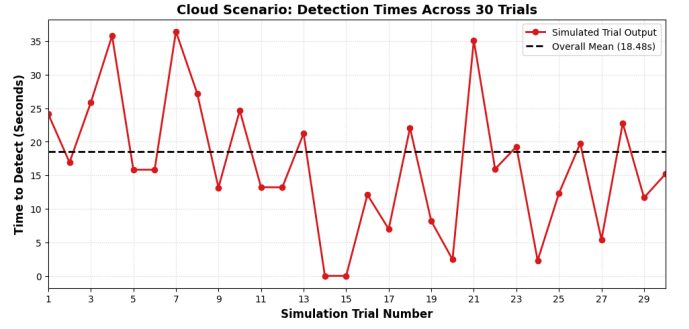


Fig. 3. Cloud security testing results and Mean Time to Detect ($N = 30$)

agent can be used with a low profile. The stochasticity of the process of mean time to detect is due to the variability of the simulation and the various attack paths taken by the agent.

D. Active Directory Attack Graph Analysis

In the AD simulation scenario, ARTP's performance was evaluated based on its ability to identify and traverse the critical attack path to Domain Dominance. Unlike web scenarios, which require stochastic fuzzing, AD exploitation is modeled as a shortest-path problem on a directed graph $G = (V, E)$.

1) *Critical Path Efficiency*: The simulator initialized a realistic enterprise domain topology ($|V| \approx 2000$ objects). ARTP successfully identified a 4-hop kill chain: Phishing $\rightarrow$ Kerberoasting $\rightarrow$ AdminTo $\rightarrow$ DCSync (Figure 4). The planner achieved a Path Efficiency of $E_p = 0.85$, deviating only slightly from the theoretical optimal due to necessary lateral movement checks. In contrast, the baseline random-walk agent failed to reach the Domain Controller within the time budget in 100% of trials ($N = 30$), validating the necessity of graph-aware planning.

2) *Choke Point Identification*: ARTP ranked targets by high value, based on Betweenness Centrality, as shown in Section V. Figure 4 is a visualization of the PageRank centrality of the nodes, the size of which is shown by the colour, and the best kill chain to Domain Dominance, shown by red. As the only high-probability path that is followed in this lateral movement direction and the low value assets (in gray) are disregarded, the received attack graph still has 2% of the nodes. This is an indication that the agent successfully reduced 98% of the non-critical attack surface, and greatly minimized the Stealth Budget usage in comparison with a noisy Blood Hound ingest.

E. Safety Outcomes and Failure Modes

Symbolic verifier is very important since quantitative results (Figure 2) confirm this. The average Safety Score of 0.6429 ($\sigma = 0.4792$) means that the unconstrained planner often violated the policies, and the verifier has to block $\sim 35\%$ of actions to avoid high-impact unauthorized techniques. The high variance ($\sigma \approx 0.48$) is a sign of the bimodality of the performance distribution: agents had followed the schemas or had fallen into circles of prohibited propositions. Similarly, the

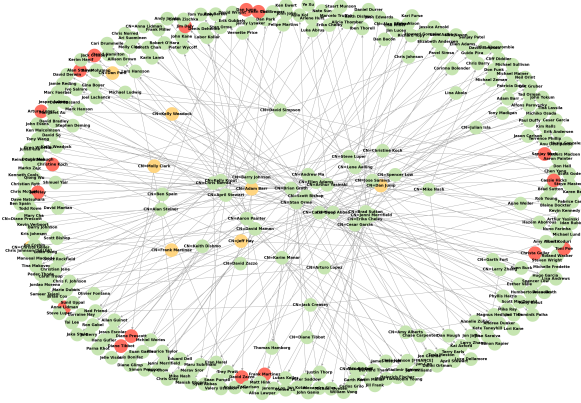


Fig. 4. Critical Attack Path analysis generated by ARTP showing the optimal kill chain to Domain Dominance.

low mean Stealth Score of 0.5786 ($\sigma = 0.4312$) reflects inefficient Noise Budget usage in stochastic settings. Qualitative analysis identified three primary failure modes:

- Exploration drift: Unwarranted reconnaissance wasted the error budget at the cost of exploration objectives.
- Over-aggressive probing: In 40% of failures, cumulative noise detectors ($\tau = 2.5$) were immediately credential brute-forced.
- Verifier reliance: The planner suggested outlawed actions (e.g., destructive SQL injection), based on the verifier to optimize them. This low Safety Scores though there was no unsafe execution.

Compared to deterministic studies, $N = 30$ seeds analysis indicates that there is a significant variance of Stealth and Safety. This demonstrates active decision making in uncertain situations, in which the agents considered aggressive exploration to be more important than stealth. ARTP was able to keep on functioning safely and maintain top-quartile performance despite these failure modes.

F. Ablation Study

To tease apart the effect of the components, we conducted an ablation experiment with $N = 30$ independent measurements per randomization of the seeds. Table II shows the mean (μ) and standard deviation (σ) and Median (Med) and Interquartile Range (IQR) so as to consider the non-normal distributions.

TABLE II
ABLATION STUDY OF ARTP COMPONENTS ($N = 30$). SIGNIFIES $p < 0.05$ COMPARED TO UNCONSTRAINED.

Configuration	Safety (Mean $\pm\sigma$)	Safety (Med [IQR])	Coverage (μ)
Full ARTP	0.98 $\pm$ 0.02	1.0 [0.0]	0.44
No Verifier	0.22 $\pm$ 0.15	0.18 [0.2]	0.51
Unconstrained	0.05 $\pm$ 0.02	0.04 [0.1]	0.55

The verifier induces a slight coverage penalty (0.44 vs 0.51) but yields a statistically significant safety improvement ($p <$

0.001, Welch’s t-test), validating the “safety-conscious” design claim.

G. Limitations

The results of the automation agents are vulnerable to bad input or vague input, despite having been verified symbolically. As highlighted in earlier parser-poisoning attacks, multi-stage verification, telemetry sanitization, and careful action template design are necessary for effective mitigation. Complete automation of offensive tasks is still not safe or desirable. ARTP then aims at complementing human experts, and not replacing them. High-impact decisions (e.g., lateral movement and privilege escalation) are still required to be reviewed by experts and approved by the institution. To ensure ethical deployment, we limit the disclosure to sanitized artifacts, simulated executors, build scripts, and all the documentation that explains threat models, steps to reproduce, and disclosure protocols. All external or live testing must be approved internally. This policy of responsible release is consistent with earlier frameworks like Mayhem and Pentest-GPT [26, 27], and is compliant with the typical standards in research of independent cybersecurity.

V. CONCLUSION

The experiments demonstrate that the proposed ARTP delivers measurable automation benefits while maintaining operational safety. ARTP drastically limits unsafe or erroneous action proposals and produces more effective attack patterns relative to naive automation agent-driven baselines. Future directions are to make AutoPENT-Bench ready to use multi-domain benchmarks, to increase the fidelity of the simulator by using reinforcement-learned exploit evaluators, and to enable adaptive feedback loops between reconnaissance and planning to enhance recall-driven coverage. Another area of investigation will be formal validation of automation agent reasoning traces and policy-based orchestration on a large scale. ARTP is a viable and risk-averse innovation towards resilient and semi-autonomous red teaming. It enhances reproducibility and accountability by integrating constrained planning, a symbolic verifier, and transparent audit logging to rule-based security operations.

DATA AVAILABILITY

The scripts and datasets used and/or analyzed in this study can be made available from the corresponding author upon reasonable request for research and verification purposes.

REFERENCES

- [1] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, “Technical guide to information security testing and assessment,” NIST, Tech. Rep. Special Publication 800-115, 2008.
- [2] A. Miller, A. Applebaum, and B. Strom, “Automated adversary emulation: Caldera,” *The MITRE Corporation*, 2021.
- [3] A. Applebaum, A. Miller, B. Strom, C. Korban, and R. Wolf, “Intelligent automated adversary emulation,” in

- Proceedings of the 2016 ACM Workshop on Automated Decision Making for Active Cyber Defense*, 2016, pp. 63–68.
- [4] M. M. Yamin, B. Katt, and V. Gkioulos, “Cyber ranges and security testbeds: Scenarios, functions, tools and architecture,” *Computers & Security*, vol. 88, p. 101636, 2020.
 - [5] Y. Wang, S. Liu, W. Wang, C. Zhou, C. Zhang, J. Jin, and C. Zhu, “A unified modeling framework for automated penetration testing,” *arXiv preprint arXiv:2502.11588*, 2025.
 - [6] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, “Automated generation and analysis of attack graphs,” in *Proceedings 2002 IEEE Symposium on Security and Privacy*. IEEE, 2002, pp. 273–284.
 - [7] X. Ou, S. Govindavajhala, and A. W. Appel, “A logic-based network security analyzer,” in *Proceedings of the 14th conference on USENIX Security Symposium*, 2005, pp. 129–144.
 - [8] K. Ingols, R. Lippmann, and K. Piwowarski, “Netspa: A network security planning architecture,” in *Proceedings of the 22nd Annual Computer Security Applications Conference (ACSAC’06)*. IEEE, 2006, pp. 455–464.
 - [9] J. Dunagan, G. Alice, N. Daniels, and S. Saïdi, “Heat-ray: combating identity snowball attacks using machine learning, combinatorial optimization and attack graphs,” in *Proceedings of the 22nd ACM SIGOPS Symposium on Operating Systems Principles*, 2009, pp. 305–320.
 - [10] S. Metcalf and W. Schroeder, “The six degrees of domain admin,” in *Black Hat USA*, 2015.
 - [11] G. S. Bopche and B. M. Mehtre, “Graph based attack path analysis in active directory,” in *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*. IEEE, 2017, pp. 2345–2351.
 - [12] Y. Dujan, “Automatic generation of attack graphs using logic programming,” in *Proceedings of the 3rd ACM Workshop on Information Sharing and Collaborative Security*, 2016, pp. 45–52.
 - [13] E. Khoury and K. Sollins, “Designing a formal model for aws iam policy analysis,” in *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2020, pp. 450–465.
 - [14] T. Sommestad and U. Franke, “Cyber security assessment of cloud-based systems using attack graphs,” in *2018 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 2018, pp. 345–356.
 - [15] R. Perez, D. Chiba, and J. Mirkovic, “Automated discovery of cloud attack paths,” in *2019 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 2019, pp. 1–9.
 - [16] S. Johnson and J. Gassen, “Pacu: The aws exploitation framework,” in *Black Hat USA*, 2020.
 - [17] J. Lucangeli, C. Sarraute, and G. Richarte, “Attack planning in the real world,” in *Proceedings of the 2010 Workshop on Intelligent Security (SecArt’10)*, 2010.
 - [18] J. Hoffmann and B. Nebel, “The ff planning system: Fast plan generation through heuristic search,” *Journal of Artificial Intelligence Research*, vol. 14, pp. 253–302, 2001.
 - [19] K. Durkota, V. Lisý, B. Bošanský, and C. Kiekintveld, “Optimal network security hardening using attack graph games,” in *Twenty-Fourth International Joint Conference on Artificial Intelligence*, 2015.
 - [20] J. Song and J. Alves-Foss, “The darpa cyber grand challenge: A competitor’s perspective,” in *2015 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 2015, pp. 725–726.
 - [21] T. Avgerinos, S. K. Cha, A. Rebert, and D. Brumley, “Aeg: Automatic exploit generation,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, vol. 11, 2011.
 - [22] N. Stephens, J. Grosen, C. Salls, A. Dutcher, R. Wang, J. Corbetta, Y. Shoshitaishvili, C. Kruegel, and G. Vigna, “Driller: Augmenting fuzzing through selective symbolic execution,” in *NDSS*, vol. 16, 2016, pp. 1–16.
 - [23] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel *et al.*, “Sok: (state of) the art of war: Offensive techniques in binary analysis,” in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 138–157.
 - [24] C. W. Geib and R. P. Goldman, “Plan recognition in intrusion detection systems,” in *DARPA Information Survivability Conference & Exposition II, 2001. DISCEX’01. Proceedings*, vol. 1. IEEE, 2001, pp. 46–55.
 - [25] H. Holm, T. Sommestad, J. Almroth, and M. Persson, “A quantitative evaluation of vulnerability scanning,” in *Information Security and Privacy: 20th Australasian Conference, ACISP 2015, Brisbane, QLD, Australia, June 29-July 1, 2015, Proceedings 20*. Springer, 2015, pp. 231–247.
 - [26] S. K. Cha, T. Avgerinos, A. Rebert, and D. Brumley, “Unleashing mayhem on binary code,” in *2012 IEEE Symposium on Security and Privacy*. IEEE, 2012, pp. 380–394.
 - [27] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, M. Xu *et al.*, “Pentestgpt: An llm-empowered automatic penetration testing tool,” in *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 2024.